

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ Malygina
Nom d'usage ▶ Thouvenin
Prénom ▶ Nadezhda
Adresse ▶ 8 impass Andre le notre 31120 Roques France

Titre professionnel visé

Développeur Web et Web M obile

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.
Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Activité-type 1 Développer la partie front-end d'une application web ou web mobile sécurisée

p.

- ▶ CP 1 Installer et configurer son environnement de travail p. 5
- ▶ CP 2 Maquetter une interface utilisateur p. 7
- ▶ CP 3 Réaliser une interface utilisateur p. 14
- ▶ CP 4 Développer la partie dynamique de l'interface utilisateur p. 18

Activité-type 2 Développer la partie back-end d'une application web ou web mobile sécurisée

p.

- ▶ CP 5 Mettre en place une base de données relationnelle p. 23
- ▶ CP 6 Développer des composants d'accès aux données..... p. 28
- ▶ CP 7 Développer des composants métier côté serveur p. 33
- ▶ CP 8 Documenter le déploiement d'une application Web ou web mobile..... p. 36

Titres, diplômes, CQP, attestations de formation *(facultatif)*

p. 39

Déclaration sur l'honneur

p. 40

Documents illustrant la pratique professionnelle *(facultatif)*

p. 41

Annexes *(Si le RC le prévoit)*

p. 42

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 ► CP 1 Installer et configurer son environnement de travail

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai préparé et configuré mon environnement de travail sur Windows 11 avec WSL 2 (Debian) afin de disposer d'un environnement Linux similaire à celui des serveurs de production.

J'ai installé PHP 8.3, Composer et Symfony CLI, puis j'ai créé mon application Kanban avec la commande `symfony new kanban --webapp`.

Cette commande installe automatiquement, via Composer, les dépendances principales d'un projet web complet, notamment Twig pour les vues, Doctrine ORM pour la gestion des données, Validator pour la validation des formulaires, et Security-CSRF pour la sécurité.

J'ai configuré Docker Compose pour exécuter une base de données PostgreSQL, connectée à mon projet grâce à la variable `DATABASE_URL` définie dans le fichier `.env.local`.

J'ai également intégré la bibliothèque Bootstrap 5 via un lien CDN dans mes fichiers Twig, afin de concevoir une interface utilisateur claire et responsive.

Enfin, j'ai initialisé un dépôt Git, créé un projet GitHub pour la gestion des versions, et j'ai travaillé dans Visual Studio Code avec des extensions dédiées à Symfony, Twig et PHP.

2. Précisez les moyens utilisés :

Système : Windows 11 + WSL 2 (Debian)

Langages : PHP 8.3, HTML5, CSS3

Frameworks et bibliothèques : Symfony 7, Twig, Bootstrap 5 (CDN), Stimulus-bundle

Base de données : PostgreSQL (conteneur Docker)

Outils : Composer, Symfony CLI, Docker, Docker Compose, Git, GitHub, Visual Studio Code

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre de formation DAWAN*

Chantier, atelier, service ► Projet personnel réalisé en cours de formation

Période d'exercice ► Du 01/09/2025 au 02/10/2025

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

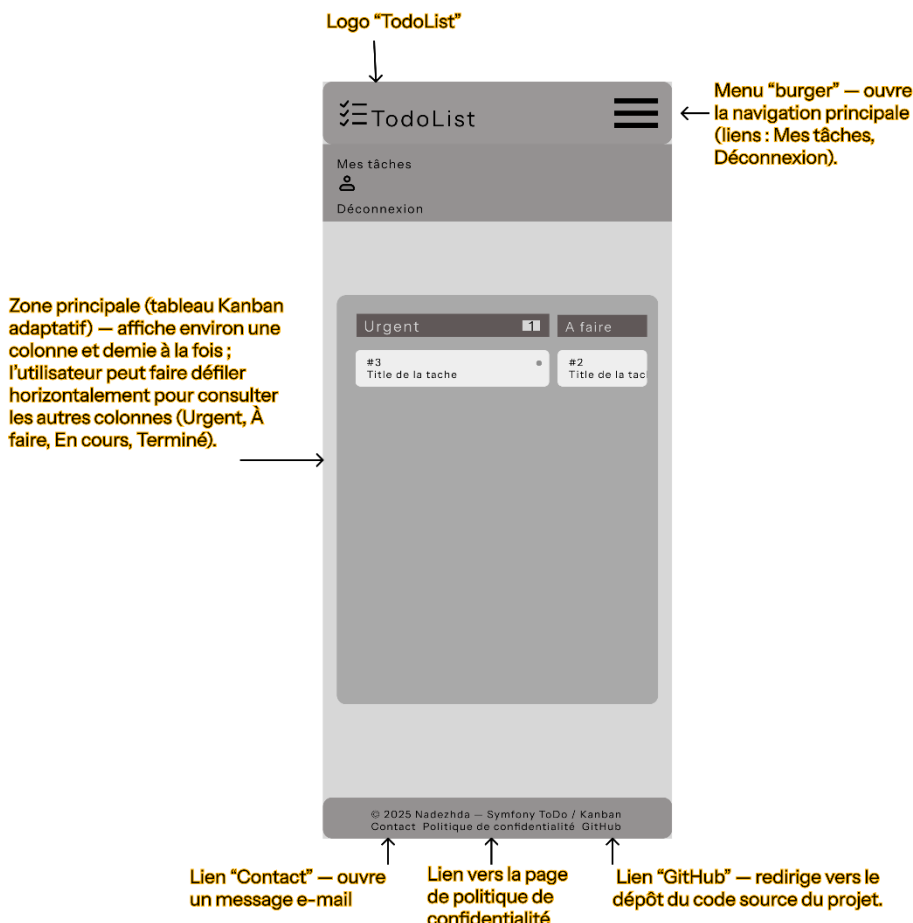
Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 ▶ CP 2 Maquetter une interface utilisateur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de la conception de mon application web de gestion de tâches (ToDoList / Kanban), j'ai réalisé plusieurs wireframes afin de définir la structure et l'ergonomie des pages principales. J'ai adopté une approche mobile-first, en commençant la conception par la version mobile, avant d'adapter les maquettes aux formats tablette et desktop. L'objectif était de garantir une interface claire, lisible et responsive sur tous les supports.



Version mobile — Page Kanban

Cette version mobile affiche uniquement deux colonnes visibles à la fois ("Urgent" et "À faire"),

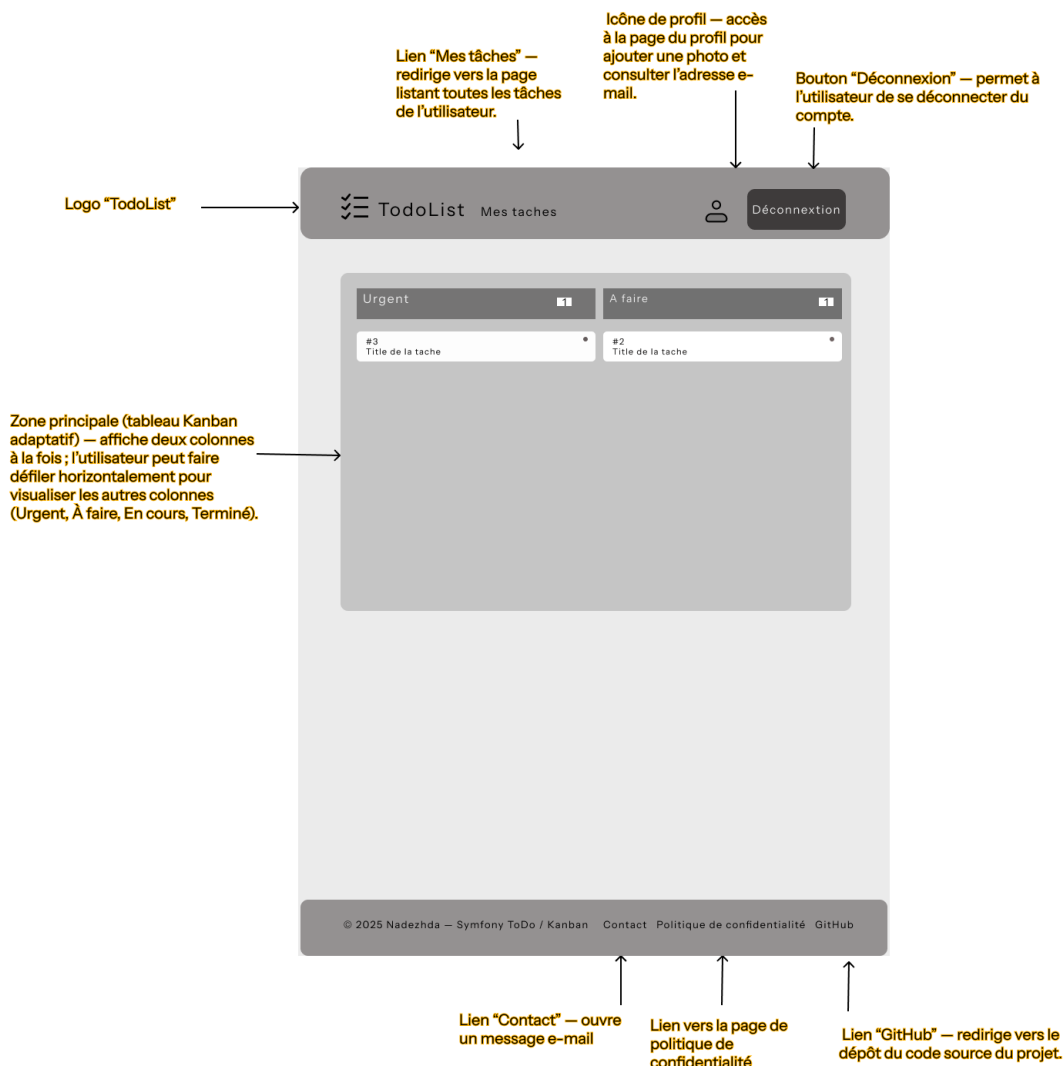
les autres colonnes ("En cours", "Terminé") apparaissent par défilement horizontal.

Le menu "burger" en haut à droite donne accès à trois sections :

le profil utilisateur, la page "Mes tâches" et la déconnexion.

Le pied de page contient les liens "Contact" , "Politique de confidentialité", "GitHub".

DOSSIER PROFESSIONNEL (DP)



Version tablette — Page Kanban

Cette version tablette affiche deux colonnes du tableau Kanban simultanément ("Urgent" et "À faire").

L'utilisateur peut faire défiler horizontalement pour visualiser les autres colonnes ("En cours" et "Terminé").

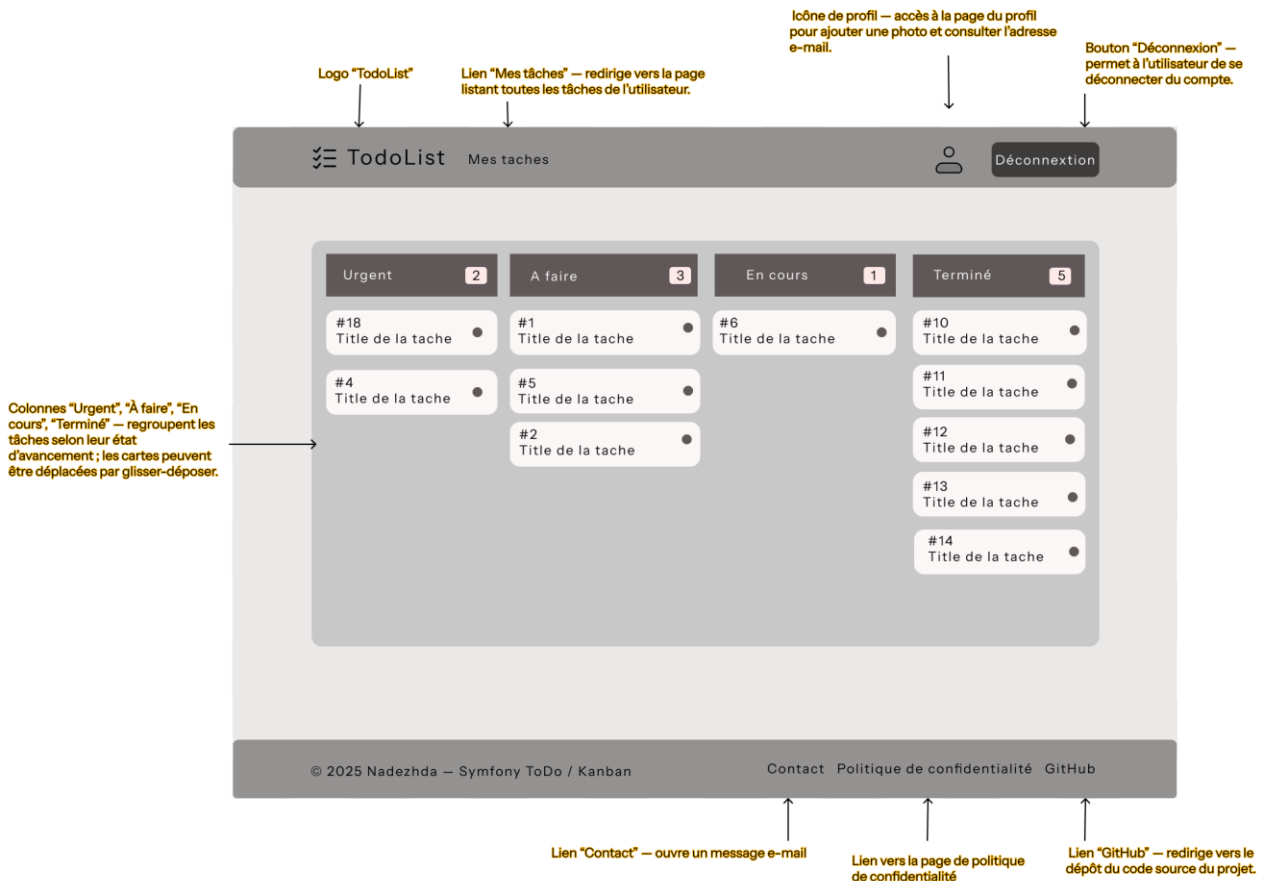
Le lien "Mes tâches" redirige vers la page listant toutes les tâches de l'utilisateur.

L'icône de profil donne accès à la page personnelle pour ajouter une photo ou consulter l'adresse e-mail.

Le bouton "Déconnexion" permet de se déconnecter du compte.

Le pied de page contient les liens "Contact" , "Politique de confidentialité", "GitHub".

DOSSIER PROFESSIONNEL (DP)



Version desktop — Page Kanban

Cette version desktop affiche simultanément les quatre colonnes du tableau Kanban ("Urgent", "À faire", "En cours", "Terminé").

Chaque tâche peut être déplacée d'une colonne à l'autre par glisser-déposer selon son état d'avancement.

Le lien "Mes tâches" redirige vers la liste complète des tâches de l'utilisateur.

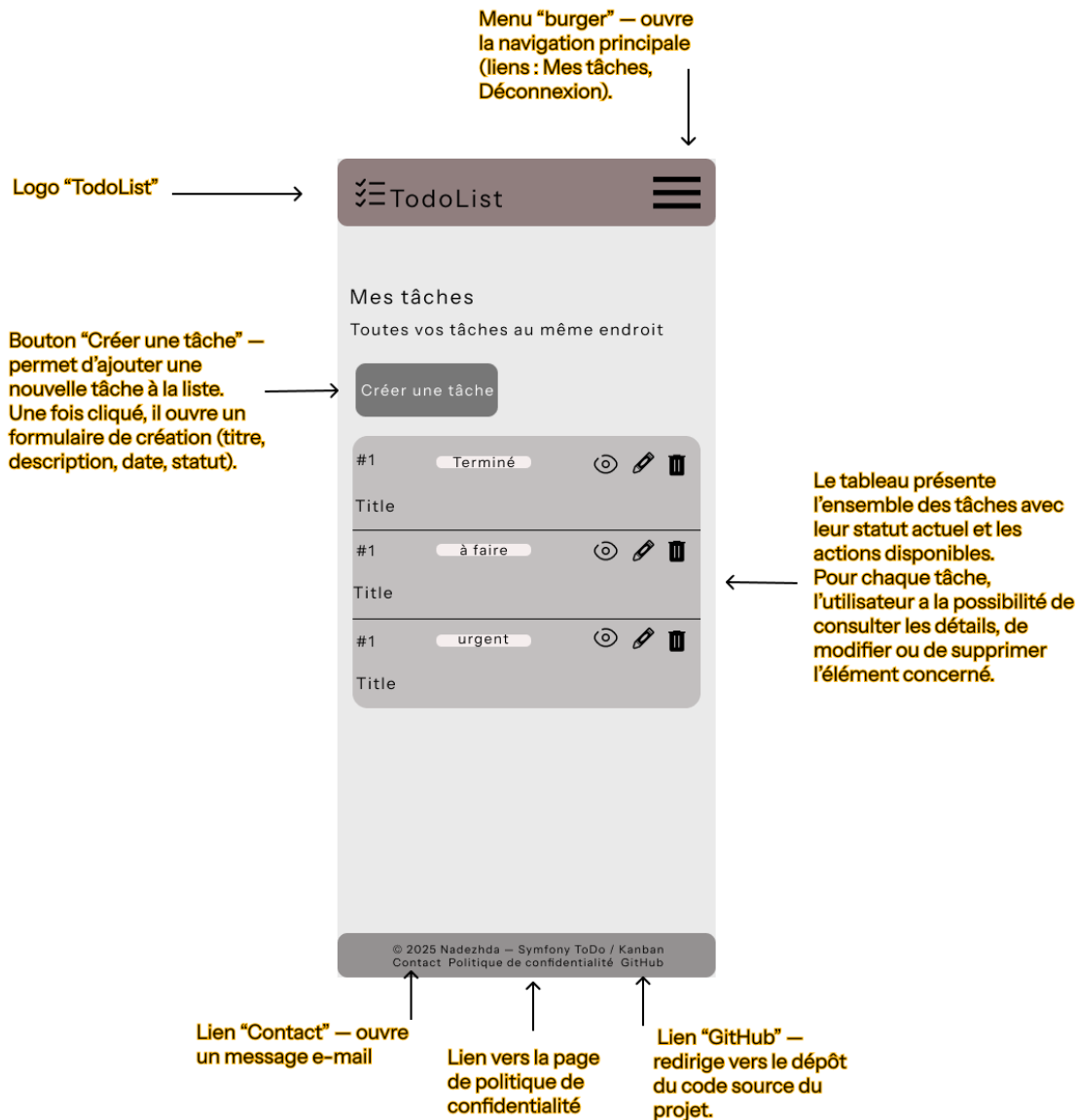
L'icône de profil permet d'accéder à la page personnelle (photo, e-mail).

Le bouton "Déconnexion" permet de se déconnecter du compte.

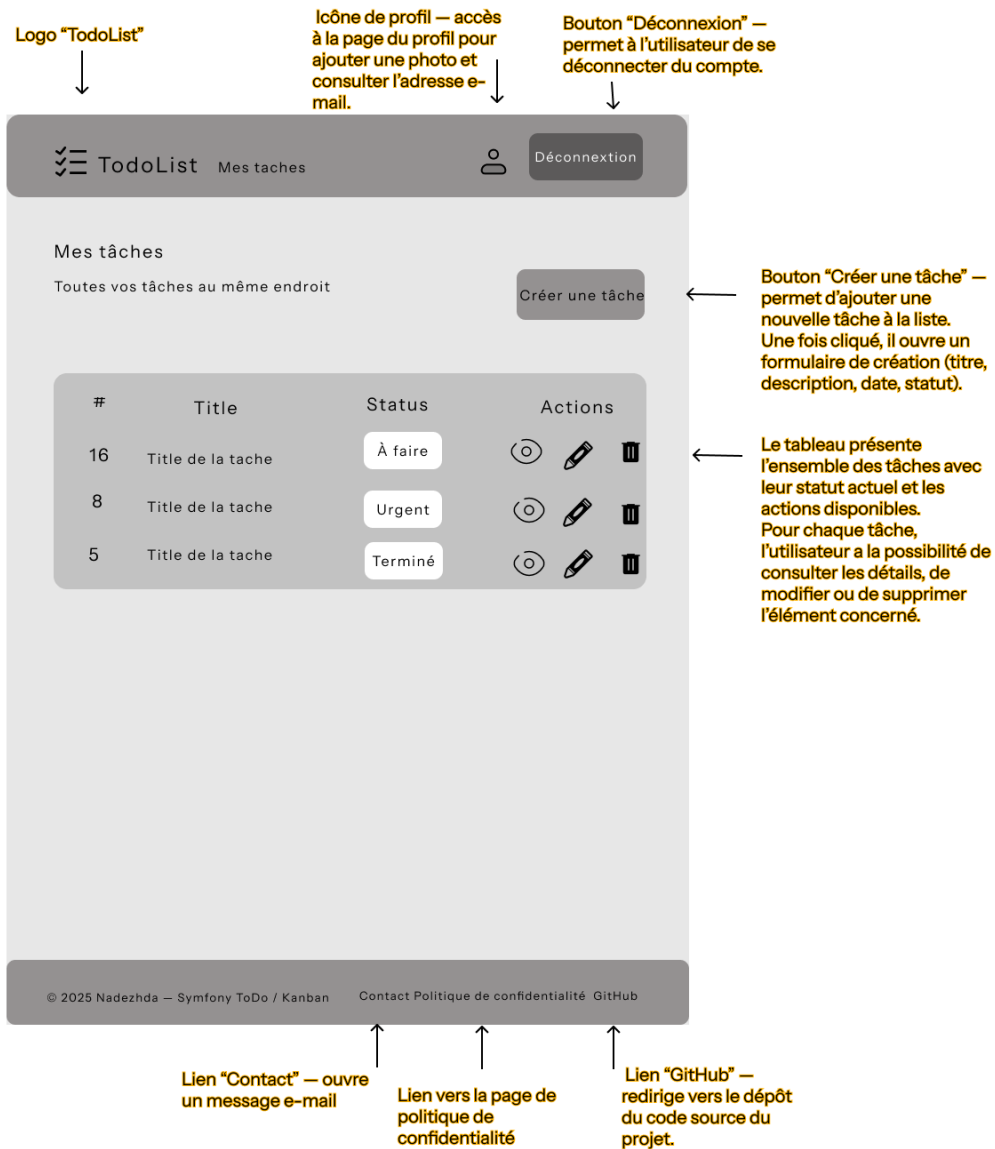
Le pied de page contient les liens "Contact", "Politique de confidentialité", "GitHub".

Autres exemples de wireframes

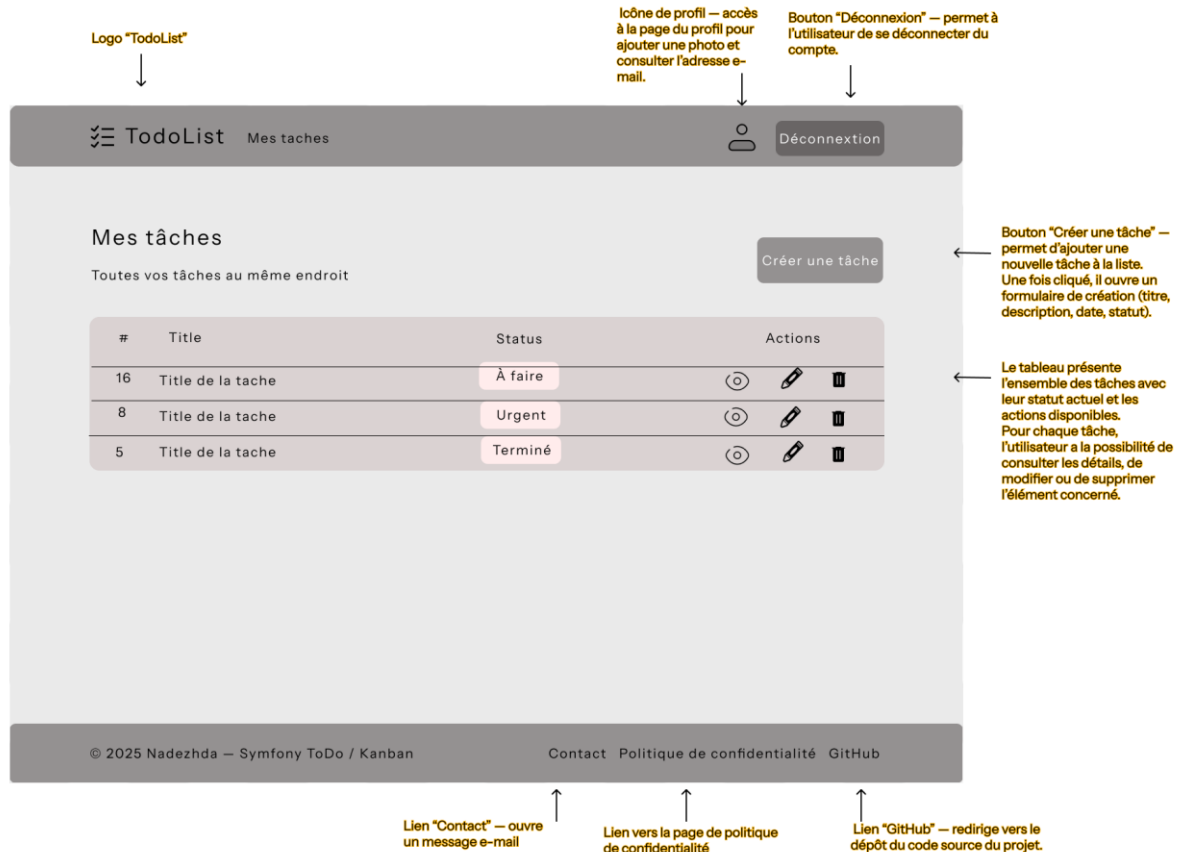
Version mobile — Page Mes tâches



Version tablette — Page Mes tâches



Version desktop — Page Mes tâches



Ces wireframes m'ont permis de vérifier que la navigation sur le site est claire et que l'utilisateur peut facilement comprendre où cliquer pour créer, voir ou modifier ses tâches.

2. Précisez les moyens utilisés :

Pour la réalisation des wireframes, j'ai utilisé l'outil Figma, qui permet de créer des maquettes interactives et de visualiser facilement les différentes tailles d'écran (mobile, tablette, desktop) .

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre de formation DAWAN*

Chantier, atelier, service ► Projet personnel réalisé en cours de formation

Période d'exercice ► Du 01/09/2025 au 02/10/2025

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°3 ► CP 3 Réaliser une interface utilisateur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

```
1 <!DOCTYPE html>
2 <html lang="{{ app.request.locale|default('fr') }}" dir="ltr">
3 <head>
4     {# SEO: autoriser uniquement la page d'accueil à être indexée #}
5     {% if app.request.attributes.get('_route') == 'app_home' %}
6         <meta name="robots" content="index, follow">
7         <meta name="description" content="Application Kanban pour organiser vos tâches facilement : créer,
8         déplacer et suivre vos projets.">
9         <meta name="keywords" content="Kanban, Symfony, gestion de tâches, application web, productivité">
10        <link rel="canonical" href="{{ absolute_url(path('app_home')) }}">
11    {% else %}
12        <meta name="robots" content="noindex, nofollow">
13    {% endif %}
14    <meta charset="UTF-8">
15    <meta name="viewport" content="width=device-width, initial-scale=1">
16
17    <title>{% block title %}Mon application{% endblock %}</title>
18    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
19    integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
20    <link rel="stylesheet" href="{{ asset('css/kanban.css') }}">
21
22    {% block javascripts %}
23        {% block importmap %}{{ importmap('app') }}{% endblock %}
24    {% endblock %}
25 </head>
26
27
28 <body data-turbo="false">
29
30     <a class="visually-hidden-focusable" href="#main-content">Aller au contenu</a>
31
32 <div class="bg-kanban">
33     {% block layout_header %}
34
35         {% include "nav/_nav.html.twig" %}
36     {% endblock %}
37
38     <main id="main-content" class="main-content" role="main" aria-live="polite">
39         {% block body %}{% endblock %}
40     </main>
41
42     {% block layout_footer %}
43
44         {% include "footer/_footer.html.twig" %}
45     {% endblock %}
46 </div>
47
48 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
49 integrity="sha384-FKyoeFForCGlyvwx9Hj09JcYn3nv7wIPV1z7YYwJrWVCXK/BmnVDxM+D2scQbITxI"
50 crossorigin="anonymous"></script>
51 </body>
52 </html>
```

1. Structure des pages .

Pour structurer toutes les pages du site et garder une même mise en page, j'utilise un gabarit commun `base.html.twig`. Ce gabarit contient l'en-tête, la zone de contenu principal et le pied de page .

- La partie `<head>` contient tout ce qui prépare l'affichage du site.

Grâce à la ligne meta viewport, le site s'adapte automatiquement à la taille de l'écran – que ce soit un téléphone, une tablette ou un ordinateur.

- Référencement (SEO) :

J'ai ajouté plusieurs balises meta pour améliorer le référencement et la description du site :

meta description, meta keywords, et meta robots.

Ces balises sont dynamiques : la page d'accueil (`app_home`) est autorisée à être indexée (`index, follow`), tandis que toutes les autres pages protégées (réservées aux utilisateurs connectés) sont exclues de l'indexation (`noindex, nofollow`).

J'ai également ajouté une balise `<link rel="canonical">` pour indiquer l'URL principale du site et éviter tout contenu dupliqué.

- Bootstrap (CDN) est utilisé pour la mise en page.

Il m'aide à créer une structure responsive avec des colonnes et des marges déjà prêtes, sans tout coder à la main.

- Le fichier `kanban.css` contient mes propres styles.

J'y ai ajouté les couleurs, les espacements et quelques ajustements visuels du tableau Kanban.

- Accessibilité :

Un lien « Aller au contenu » est présent au tout début de la page.

Il permet aux personnes utilisant un lecteur d'écran ou un clavier d'accéder directement au contenu principal sans devoir passer par le menu.

- La partie `<main>` (avec `role="main"`) correspond à la zone centrale du site, là où s'affiche le contenu principal.

L'attribut `aria-live="polite"` aide les lecteurs d'écran à annoncer les changements de contenu sans interrompre la lecture en cours.

- En-tête et pied de page communs :

Le menu de navigation (nav/_nav.html.twig) et le pied de page (footer/_footer.html.twig) sont inclus automatiquement dans toutes les pages.

Cela me permet de garder une même présentation sur tout le site.

- Bloc de contenu principal :

La ligne `{% block body %}` est un espace réservé où chaque page spécifique vient insérer son propre contenu (par exemple la page du Kanban).

De la même manière, `{% block title %}` permet de changer le titre selon la page.

- Les scripts :

Le JavaScript de Bootstrap est chargé à la fin du document, pour que les composants comme les menus déroulants ou les boutons fonctionnent correctement.

2. Responsive de la page Kanban.

Extrait de code (largeurs des colonnes) :

```
1 <div class="row flex-nowrap g-3 overflow-auto">
2   <div class="col-10 col-sm-8 col-md-6 col-lg-3">...</div>
3   <div class="col-10 col-sm-8 col-md-6 col-lg-3">...</div>
4   <div class="col-10 col-sm-8 col-md-6 col-lg-3">...</div>
5   <div class="col-10 col-sm-8 col-md-6 col-lg-3">...</div>
6 </div>
7 |
```

Bootstrap gère toute l'adaptabilité automatiquement.

- Sur petits écrans (col-10 / col-sm-8), on voit une seule colonne et un petit aperçu de la suivante grâce au défilement horizontal.
- Sur tablette (col-md-6), deux colonnes sont affichées côte à côte.
- Sur ordinateur (col-lg-3), les quatre colonnes apparaissent sur une seule ligne.
- La classe `g-3` crée un espace régulier entre les colonnes.

L'interface du tableau Kanban est responsive grâce à la grille Bootstrap : 1 colonne défilante sur mobile, 2 sur tablette, 4 sur desktop. Aucune media query manuelle n'a été nécessaire.

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Pour rendre la page Kanban responsive, j'ai utilisé Bootstrap 5 et ses classes de grille (col-sm, col-md, col-lg). J'ai vérifié l'affichage sur différents écrans grâce aux outils de développement du navigateur (onglet Responsive Design). Je me suis également appuyée sur la documentation MDN (Mozilla Developer Network) pour bien utiliser les balises meta et améliorer l'accessibilité. Enfin, j'ai testé les performances et le référencement (SEO) avec Lighthouse.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► **Centre de formation DAWAN**

Chantier, atelier, service ► **Projet personnel réalisé en cours de formation**

Période d'exercice ► **Du 01/09/2025 au 02/10/2025**

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°4 ► CP 4 Développer la partie dynamique de l'interface utilisateur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

```
1 <script>
2 document.addEventListener('DOMContentLoaded', () => {
3   const columns = document.querySelectorAll('.kanban-cards');
4   let dragged = null;
5   const DOT_BY_COL = { todo: 'bg-primary', doing: 'bg-warning', done: 'bg-success', urgent: 'bg-danger' };
6
7
8   function applyDotColor(task, colKey) {
9     const dot = task.querySelector('.dot');
10    if (!dot) return;
11    dot.classList.remove('bg-primary', 'bg-warning', 'bg-success', 'bg-danger');
12    dot.classList.add(DOT_BY_COL[colKey] || 'bg-secondary');
13  }
14
15
16
17   function bindDraggableCards() {
18     document.querySelectorAll('.kanban-card').forEach(card => {
19
20       card.setAttribute('draggable', 'true');
21
22       card.addEventListener('dragstart', (e) => {
23         dragged = card;
24         card.classList.add('dragging');
25         e.dataTransfer.setData('text/plain', card.dataset.id);
26       });
27
28       card.addEventListener('dragend', () => {
29         dragged?.classList.remove('dragging');
30         dragged = null;
31       });
32     });
33   }
34   bindDraggableCards();
35
36   columns.forEach(col => {
37     col.addEventListener('dragover', (e) => {
38       e.preventDefault();
39       if (!dragged) return;
40       col.appendChild(dragged);
41     });
42
43     function updateCounters() {
44       ['todo', 'doing', 'done', 'urgent'].forEach(k => {
45         const box = document.getElementById('col-' + k);
46         const badge = document.getElementById('count-' + k);
47         if (!box || !badge) return;
48         badge.textContent = box.querySelectorAll('.kanban-card').length;
49       });
50     }
51   })
52 }
```

Ce code JavaScript rend le tableau Kanban entièrement interactif.

Il permet de déplacer les tâches d'une colonne à une autre sans recharger la page. Quand une carte change de colonne, sa couleur (le petit point) s'adapte automatiquement, et le nombre de cartes dans chaque colonne est mis à jour en direct. Tout cela rend l'interface vivante et dynamique.

■ Bloc 1 — Début et variables générales

(lignes 1–6)

- `DOMContentLoaded` : le script démarre quand toute la page HTML est chargée.
- `columns = document.querySelectorAll('.kanban-cards')` : je récupère toutes les colonnes du tableau Kanban.
- `dragged = null` : cette variable garde en mémoire la carte qu'on est en train de déplacer.
- `DOT_BY_COL = {...}` : chaque colonne correspond à une couleur (le petit point sur la carte change selon la colonne).

■ Bloc 2 — Fonction qui change la couleur du point

`applyDotColor(task, colKey)` (lignes 8–14)

- La fonction cherche dans la carte l'élément `.dot`.
- Elle enlève les anciennes couleurs et ajoute la nouvelle, selon la colonne (`todo`, `doing`, `done`, `urgent`).
- Résultat : quand on déplace la carte, la couleur du point change immédiatement, sans recharger la page.

■ Bloc 3 — Rendre les cartes déplaçables

`bindDraggableCards()` (lignes 17–34)

- Je sélectionne toutes les cartes `.kanban-card`.
- `setAttribute('draggable', 'true')` : j'active la fonction de glisser-déposer du navigateur.
- Événement `dragstart` :
 - Je garde la carte dans la variable `dragged`.
 - J'ajoute une classe `dragging` (pour la mettre en surbrillance).
 - Je garde aussi son id dans la mémoire du navigateur (`dataTransfer`).
- Événement `dragend` :
 - Je retire la classe `dragging`.
 - Je remets `dragged = null`.
- Ensuite, j'appelle `bindDraggableCards()` pour que tout fonctionne dès le chargement.

■ Bloc 4 — Autoriser le dépôt dans les colonnes

`columns.forEach(... dragover ...)` (lignes 36–41)

- Pour chaque colonne, j'ajoute un écouteur `dragover`.
- `e.preventDefault()` est obligatoire, sinon le navigateur n'autorise pas le dépôt.

- Si une carte est déplacée, je fais `col.appendChild(dragged)` : la carte se place directement dans la nouvelle colonne, sans rechargement de la page.

■ Bloc 5 — Mettre à jour les compteurs

`updateCounters()` (lignes 43–51)

- Pour chaque colonne (todo, doing, done, urgent) :
- Je trouve le conteneur (`#col-...`) et le badge (`#count-...`).
- Je compte le nombre de cartes à l'intérieur.
- Cette fonction est appelée après chaque dépôt pour que les nombres s'actualisent automatiquement à l'écran.

```
52     updateCounters();
53
54     col.addEventListener('drop', () => {
55
56         const colKey = (col.id || '').replace('col-', '');
57         if (dragged) applyDotColor(dragged, colKey);
58         updateCounters();
59
60         saveOrder();
61     });
62
63
64
65
66 function saveOrder() {
67     const csrf = document.querySelector('meta[name="csrf-kanban"]')?.content || '';
68
69     const data = {
70         _token: csrf,
71         todo: Array.from(document.querySelectorAll('#col-todo .kanban-card'))
72             .map(c => c.dataset.id),
73         doing: Array.from(document.querySelectorAll('#col-doing .kanban-card'))
74             .map(c => c.dataset.id),
75         done: Array.from(document.querySelectorAll('#col-done .kanban-card'))
76             .map(c => c.dataset.id),
77         urgent: Array.from(document.querySelectorAll('#col-urgent .kanban-card'))
78             .map(c => c.dataset.id),
79     };
80
81
82     fetch('/kanban/save-order', {
83         method: 'POST',
84         headers: { 'Content-Type': 'application/json' },
85         body: JSON.stringify(data)
86     });
87 }
88
89 </script>
90
```

Dans cette deuxième partie du code, quand une carte est déposée, le nombre de tâches se met à jour, la couleur change, et le nouvel ordre est sauvegardé sur le serveur via `fetch()`, sans rechargement de la page.

■ Bloc 6 — Événement « drop » (lignes 55 – 63)

- Ensuite, un événement drop est ajouté sur chaque colonne.
- Il se déclenche quand l'utilisateur lâche une carte.
- Le script récupère la clé de la colonne (`colKey`) pour savoir où la carte a été déposée.
- Si une carte est bien en train d'être déplacée :

- la fonction `applyDotColor(dragged, colKey)` change la couleur du petit point sur la carte selon la colonne ;
- `updateCounters()` est appelée à nouveau pour actualiser le nombre de cartes ;
- puis `saveOrder()` envoie le nouvel ordre des cartes au serveur.
- Tout cela se fait instantanément et sans recharger la page.

Bloc 7 — Fonction `saveOrder()` (lignes 66 – 88)

- Le code récupère d'abord le jeton CSRF dans la balise `<meta name="csrf-kanban">` pour sécuriser la requête.
- Ensuite, il crée un objet `data` qui contient :
 - `_token` : le jeton CSRF ;
 - quatre tableaux (un pour chaque colonne : `todo`, `doing`, `done`, `urgent`) avec les ID des cartes présentes dans chaque colonne.
 - Ces ID sont pris directement dans le DOM, grâce à `document.querySelectorAll()` (pour trouver les cartes),
`Array.from()` (pour créer un tableau) et `.map()` (pour extraire l'ID de chaque carte).
- Enfin, les données sont envoyées au serveur avec `fetch()` :
 - méthode `POST` ;
 - en-tête `'Content-Type': 'application/json'` ;
 - le corps contient l'objet `data` transformé en JSON.
- Le nouvel ordre des cartes est donc enregistré sur le serveur, sans recharger la page :
l'utilisateur déplace une carte, et tout est mis à jour tout de suite.

2. Précisez les moyens utilisés :

Pour réaliser cette partie dynamique, je me suis appuyée sur la documentation de MDN (Mozilla Developer Network) pour comprendre le fonctionnement des événements `drag and drop` en JavaScript. J'ai également consulté des vidéos pédagogiques sur YouTube et utilisé ChatGPT pour clarifier certains points techniques, comme la fonction `fetch()` et la gestion du token CSRF.

Ensuite, j'ai testé et ajusté le code dans la console du navigateur, en utilisant des messages `console.log()` pour vérifier que tout fonctionnait correctement, sans erreur et sans rechargement de page.

DOSSIER PROFESSIONNEL (DP)

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre de formation DAWAN*

Chantier, atelier, service ► Projet personnel réalisé en cours de formation

Période d'exercice ► Du 01/09/2025 au 02/10/2025

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°1 ► CP 5 Mettre en place une base de données relationnelle

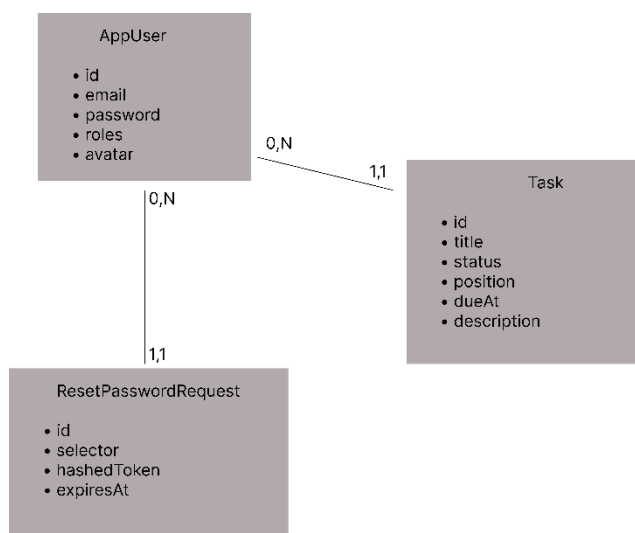
1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

« Le MCD »

J'ai réalisé le Modèle Conceptuel de Données (MCD) afin de définir la structure logique de ma base de données.

Ce modèle me permet de visualiser les principales entités de mon application, leurs attributs et les liens qui existent entre elles.

Il constitue une étape essentielle avant la mise en place du modèle logique (MLD) et la création physique de la base relationnelle.



Choix des cardinalités

Les cardinalités ont été définies en fonction des règles de fonctionnement de l'application.

Un utilisateur peut exister dans le système sans avoir encore créé de tâche ou sans avoir effectué de demande de réinitialisation de mot de passe : les cardinalités sont donc (0,N) du côté de l'utilisateur.

En revanche, chaque tâche et chaque demande de réinitialisation de mot de passe sont nécessairement rattachées à un seul utilisateur existant : les cardinalités sont donc (1,1) du côté de ces entités.

Après avoir défini la représentation visuelle de ma base de données avec le MCD, j'ai créé un MLD (Modèle Logique de Données) pour représenter les relations entre les tables.

« Le MLD »

APP_USER (id, email, password, roles, avatar)

TASK (id, title, status, position, due_at, description, #owner_id)

RESET_PASSWORD_REQUEST (id, selector, hashed_token, expires_at, #user_id)

« Le dictionnaire des données »

J'ai ensuite créé un dictionnaire des données pour décrire les champs des tables, leurs types, contraintes et rôles dans la base.

DOSSIER PROFESSIONNEL (DP)

Table app_user

FIELD	TYPE	DETAILS	DESCRIPTION
id	int	PK, NOT NULL, GENERATED AS IDENTITY	Identifiant unique de l'utilisateur
email	varchar(180)	UNIQUE, NOT NULL	Adresse e-mail de l'utilisateur
roles	json	NOT NULL	Rôles stockés au format JSON
password	varchar(255)	NOT NULL	Mot de passe chiffré
avatar	varchar(255)	DEFAULT NULL	Nom de fichier de l'avatar téléchargé

Table task

FIELD	TYPE	DETAILS	DESCRIPTION
id	int	PK, NOT NULL, GENERATED AS IDENTITY	Identifiant unique de la tâche
title	varchar(120)	NOT NULL	Titre de la tâche
status	varchar(20)	NOT NULL	Statut de la tâche (todo, doing, done, urgent)
position	int	NOT NULL, DEFAULT 0	Position d'affichage (ordre dans la colonne)
due_at	TIMESTAMP WITHOUT TIME ZONE	DEFAULT NULL	Date d'échéance de la tâche
description	text	DEFAULT NULL	Description optionnelle de la tâche
owner_id	int	NOT NULL, FK → app_user(id)	Référence à l'utilisateur propriétaire

Table reset_password_request

FIELD	TYPE	DETAILS	DESCRIPTION
id	int	PK, NOT NULL, GENERATED AS IDENTITY	Identifiant unique de la demande
user_id	int	NOT NULL, FK → app_user(id)	Utilisateur ayant demandé la réinitialisation
selector	varchar(20)	NOT NULL	Identifiant public du lien de réinitialisation
hashed_token	varchar(100)	NOT NULL	Jeton privé haché (stocké côté serveur)
requested_at	TIMESTAMP WITHOUT TIME ZONE	NOT NULL	Date et heure de la demande
expires_at	TIMESTAMP WITHOUT TIME ZONE	NOT NULL	Date d'expiration du lien

« Le MPD »

Après avoir réalisé le dictionnaire des données, j'ai créé le MPD (Modèle Physique de Données) à partir de ma base de données réelle, en exportant le script SQL via PgAdmin.

Afin de ne pas surcharger le document, j'ai conservé uniquement les tables principales de l'application : app_user, task et reset_password_request.

Les éléments techniques ajoutés automatiquement par PostgreSQL ou par les bibliothèques (Doctrine, Messenger, fonctions et triggers de notification, etc.) ont été volontairement retirés, car ils ne figurent pas dans le MLD et ne concernent pas les données métier.

```
2  -- MPD (extrait nettoyé) - uniquement les tables fonctionnelles
3  -- Tables: app_user, task, reset_password_request
4  -- PK / FK / INDEX / UNIQUE conservés
5
6  -- =====
7  -- Séquences
8  -- =====
9  CREATE SEQUENCE IF NOT EXISTS public.app_user_id_seq
10 |   AS integer START WITH 1 INCREMENT BY 1 NO MINVALUE NO MAXVALUE CACHE 1;
11
12  CREATE SEQUENCE IF NOT EXISTS public.task_id_seq
13 |   AS integer START WITH 1 INCREMENT BY 1 NO MINVALUE NO MAXVALUE CACHE 1;
14
15  CREATE SEQUENCE IF NOT EXISTS public.reset_password_request_id_seq
16 |   AS integer START WITH 1 INCREMENT BY 1 NO MINVALUE NO MAXVALUE CACHE 1;
17
18  -- =====
19  -- Tables
20  -- =====
21  CREATE TABLE IF NOT EXISTS public.app_user (
22 |   id integer NOT NULL,
23 |   email character varying(180) NOT NULL,
24 |   roles json NOT NULL,
25 |   password character varying(255) NOT NULL,
26 |   avatar character varying(255) DEFAULT NULL::character varying
27 | );
28
29  CREATE TABLE IF NOT EXISTS public.task (
30 |   id integer NOT NULL,
31 |   title character varying(120) NOT NULL,
32 |   status character varying(20) NOT NULL,
33 |   "position" integer DEFAULT 0 NOT NULL,
34 |   due_at timestamp(0) without time zone DEFAULT NULL::timestamp without time zone,
35 |   owner_id integer NOT NULL,
36 |   description text
37 | );
38
39  CREATE TABLE IF NOT EXISTS public.reset_password_request (
40 |   id integer NOT NULL,
41 |   user_id integer NOT NULL,
42 |   selector character varying(20) NOT NULL,
43 |   hashed_token character varying(100) NOT NULL,
44 |   requested_at timestamp(0) without time zone NOT NULL,
45 |   expires_at timestamp(0) without time zone NOT NULL
46 | );
```

```
48 -- =====
49 -- Defaults (liens séquence -> colonne)
50 -- =====
51 ALTER TABLE ONLY public.app_user
52 | ALTER COLUMN id SET DEFAULT nextval('public.app_user_id_seq'::regclass);
53
54 ALTER TABLE ONLY public.task
55 | ALTER COLUMN id SET DEFAULT nextval('public.task_id_seq'::regclass);
56
57 ALTER TABLE ONLY public.reset_password_request
58 | ALTER COLUMN id SET DEFAULT nextval('public.reset_password_request_id_seq'::regclass);
59
60 -- =====
61 -- Clés primaires
62 -- =====
63 ALTER TABLE ONLY public.app_user
64 | ADD CONSTRAINT app_user_pkey PRIMARY KEY (id);
65
66 ALTER TABLE ONLY public.task
67 | ADD CONSTRAINT task_pkey PRIMARY KEY (id);
68
69 ALTER TABLE ONLY public.reset_password_request
70 | ADD CONSTRAINT reset_password_request_pkey PRIMARY KEY (id);
71
72 -- =====
73 -- Index / Unique
74 -- =====
75 CREATE UNIQUE INDEX IF NOT EXISTS uniq_app_user_email ON public.app_user USING btree (email);
76 CREATE INDEX IF NOT EXISTS idx_task_owner_id ON public.task USING btree (owner_id);
77 CREATE INDEX IF NOT EXISTS idx_rpr_user_id ON public.reset_password_request USING btree (user_id);
78
79 -- =====
80 -- Clés étrangères
81 -- =====
82 ALTER TABLE ONLY public.task
83 | ADD CONSTRAINT fk_task_owner FOREIGN KEY (owner_id) REFERENCES public.app_user(id);
84
85 ALTER TABLE ONLY public.reset_password_request
86 | ADD CONSTRAINT fk_rpr_user FOREIGN KEY (user_id) REFERENCES public.app_user(id);
```

2. Précisez les moyens utilisés :

Pour la conception des différents schémas (MCD, MLD) j'ai utilisé Figma, ce qui m'a permis de représenter visuellement les entités, les relations et les échanges de données.

Pour la réalisation du MPD (Modèle Physique de Données) et l'export du script SQL, j'ai utilisé pgAdmin avec la base de données PostgreSQL.

DOSSIER PROFESSIONNEL (DP)

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► **Centre de formation DAWAN**

Chantier, atelier, service ► **Projet personnel réalisé en cours de formation**

Période d'exercice ► Du **01/09/2025** au **02/10/2025**

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°2 ▶ CP 6 Développer des composants d'accès aux données

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Initialisation de la connexion à la base de données

Dans le fichier .env, j'ai défini l'adresse de ma base de données :

```
DATABASE_URL="postgresql://app:*****@127.0.0.1:5432/symfony_todo?charset=utf8"
```

Cette variable contient toutes les informations de connexion :

- app = utilisateur PostgreSQL,
- 127.0.0.1:5432 = adresse et port du serveur,
- symfony_todo = nom de la base,
- charset=utf8 = encodage des échanges.

Dans config/packages/doctrine.yaml, j'ai activé la ligne suivante :

```
url: "%env(resolve:DATABASE_URL)%"
```

Cela permet à Symfony de récupérer automatiquement la variable DATABASE_URL depuis le fichier .env.

Toujours dans doctrine.yaml, la section DBAL contient :

```
driver: "pdo_pgsql"  
server_version: "16"
```

Cela permet à Symfony de transmettre automatiquement à Doctrine la variable DATABASE_URL issue du fichier .env,

afin que Doctrine puisse établir la connexion à la base de données.

Toujours dans doctrine.yaml, la section ORM relie Doctrine aux entités du projet :

mappings:

App:

```
type: attribute  
dir: "%kernel.project_dir%/src/Entity"  
prefix: 'App\Entity'
```

Doctrine lit mes classes PHP annotées (AppUser, Task, ResetPasswordRequest) dans le dossier src/Entity et les mappe automatiquement aux tables PostgreSQL.

```
8
9 #[ORM\Entity(repositoryClass: TaskRepository::class)]
10 #[ORM\Table(name: 'task')]
11 class Task
12 {
13     #[ORM\Id]
14     #[ORM\GeneratedValue]
15     #[ORM\Column]
16     private ?int $id = null;
17
18     #[ORM\Column(length: 120)]
19     private ?string $title = null;
20
21     #[ORM\Column(length: 20)]
22     private ?string $status = null;
23
24     #[ORM\Column(type: 'integer', options: ['default' => 0])]
25     private int $position = 0;
26
27     #[ORM\Column(type: 'datetime_immutable', nullable: true)]
28     private ?\DateTimeImmutable $dueAt = null;
29
30     #[ORM\ManyToOne(targetEntity: AppUser::class)]
31     #[ORM\JoinColumn(nullable: false)]
32     private ?AppUser $owner = null;
33
34     #[ORM\Column(type: Types::TEXT, nullable: true)]
35     private ?string $description = null;
36 }
```

Exemple concret — comment j'accède aux données avec Doctrine ORM

Entité support : Task

Les attributs #[ORM\...] de l'entité décrivent la table et ses colonnes.

Les requêtes SQL ne sont pas écrites à la main : elles sont générées automatiquement à partir des attributs de mon entité Task (#[ORM\Entity], #[ORM\Column], relations ManyToOne, etc.). Doctrine DBAL ouvre la connexion PostgreSQL et exécute ces requêtes.

Mon code d'exemple est dans TaskController

Lecture (READ) — lister les tâches

```
#[Route(name: 'app_task_index', methods: ['GET'])]
public function index(TaskRepository $taskRepository): Response
{
    $tasks = $taskRepository->findBy(['owner' => $this->getUser()], ['id' => 'ASC']);
    return $this->render('task/index.html.twig', [
        'tasks' => $tasks
    ]);
}
```

findBy(['owner' => \$this->getUser()], ['id' => 'ASC'])

- construit une requête SQL SELECT ... WHERE owner = utilisateur connecté.

- Doctrine exécute la requête et hydrate

chaque ligne en objet Task appartenant uniquement à ce user. Le tableau des tâches de l'utilisateur connecté est ensuite envoyé au template Twig.

Création (CREATE) — ajouter une tâche

```
24
25 #[Route('/new', name: 'app_task_new', methods: ['GET', 'POST'])]
26 public function new(Request $request, EntityManagerInterface $entityManager): Response
27 {
28     $task = new Task();
29     $form = $this->createForm(TaskType::class, $task);
30     $form->handleRequest($request);
31
32     if ($form->isSubmitted() && $form->isValid()) {
33
34         /** @var \App\Entity\AppUser $user */
35         $user = $this->getUser();
36         $task->setOwner($user);
37
38         $entityManager->persist($task);
39         $entityManager->flush();
40
41         return $this->redirectToRoute('app_task_index', [], Response::HTTP_SEE_OTHER);
42     }
43
44     return $this->render('task/new.html.twig', [
45         'task' => $task,
46         'form' => $form,
47     ]);
48 }
49
```

- `persist()` dit à Doctrine « insérer cet objet ».
- `flush()` émet l'INSERT (avec title, status, position, owner_id, etc.).
- La relation `ManyToOne` owner remplit la FK automatiquement.

Modification (UPDATE) — éditer une tâche

```
63
64 #[Route("/{id}/edit', name: 'app_task_edit', methods: ['GET', 'POST'])]
65 public function edit(Request $request, Task $task, EntityManagerInterface $entityManager): Response
66 {
67     $form = $this->createForm(TaskType::class, $task);
68     $form->handleRequest($request);
69
70     if ($task->getOwner() !== $this->getUser()) {
71         throw $this->createAccessDeniedException("Vous n'avez pas accès à cette tâche.");
72     }
73
74     if ($form->isSubmitted() && $form->isValid()) {
75         $entityManager->flush();
76
77         return $this->redirectToRoute('app_task_index', [], Response::HTTP_SEE_OTHER);
78     }
79
80     return $this->render('task/edit.html.twig', [
81         'task' => $task,
82         'form' => $form,
83     ]);
84 }
85
```

- Symfony récupère automatiquement la tâche à partir de l'id présent dans l'URL
- Doctrine garde en mémoire l'état de la tâche avant modification
- Lors du `flush()`, Doctrine met à jour automatiquement la tâche en base si des champs ont changé

Suppression (DELETE) — supprimer une tâche

```
85
86 #[Route('/{id}', name: 'app_task_delete', methods: ['POST'])]
87 public function delete(Request $request, Task $task, EntityManagerInterface $entityManager): Response
88 {
89     if ($task->getOwner() !== $this->getUser()) {
90         throw $this->createAccessDeniedException("Vous n'avez pas accès à cette tâche.");
91     }
92     if ($this->isCsrfTokenValid('delete', $task->getId(), $request->getPayload()->getString('_token'))) {
93         $entityManager->remove($task);
94         $entityManager->flush();
95     }
96
97     return $this->redirectToRoute('app_task_index', [], Response::HTTP_SEE_OTHER);
98 }
99 }
```

- CSRF vérifié avant l'action.
- `remove()` + `flush()` → DELETE FROM task WHERE id =

Ce que fait Doctrine :

- Doctrine analyse les attributs ORM de mon entité Task pour savoir dans quelle table enregistrer ou lire les données, quelles colonnes utiliser et comment appliquer les relations (clés étrangères).
- Il génère automatiquement la requête SQL appropriée (SELECT, INSERT, UPDATE ou DELETE) selon l'action effectuée dans le code.
- Le composant DBAL s'occupe de la connexion avec PostgreSQL (via PDO) et de l'exécution de la requête.
- Doctrine hydrate les résultats, c'est-à-dire qu'il convertit les lignes SQL en objets PHP (Task, AppUser, etc.) et inversement lors de la sauvegarde.
- Quand j'appelle `flush()`, Doctrine regroupe toutes les modifications dans une seule transaction, ce qui rend l'opération rapide, cohérente et fiable.
- Enfin, Doctrine protège automatiquement les paramètres des requêtes grâce à un système de liaison sécurisée (`setParameter()`), ce qui évite les injections SQL sans que le développeur n'ait besoin de gérer la sécurité manuellement.

2. Précisez les moyens utilisés :

1. Framework : Symfony 7
2. ORM : Doctrine ORM et Doctrine DBAL
3. Langage : PHP 8.2
4. Base de données : PostgreSQL 16
5. Outil de mapping : Attributs `#[ORM\Entity]`, `#[ORM\Column]`, `#[ORM\ManyToOne]`
6. Requêtes : Méthodes `findAll()`, `persist()`, `remove()`, `flush()`
7. Sécurité : Liaison automatique des paramètres avec `setParameter()` (protection contre les injections SQL)
8. Structure MVC : Contrôleurs, entités et repositories séparés
9. Outil de développement : Visual Studio Code, Symfony CLI, Postgres, pgAdmin

DOSSIER PROFESSIONNEL (DP)

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► **Centre de formation DAWAN**

Chantier, atelier, service ► **Projet personnel réalisé en cours de formation**

Période d'exercice ► **Du 01/09/2025 au 02/10/2025**

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°3 ► CP 7 Développer des composants métier côté serveur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour enregistrer les changements de statut et de position des tâches dans mon application, j'ai développé un endpoint serveur POST /kanban/save-order.

```
15 final class KanbanController extends AbstractController
43 #[Route('/kanban/save-order', name: 'kanban_save_order', methods: ['POST'])]
44 public function saveOrder(
45     Request $req,
46     TaskRepository $repo,
47     EntityManagerInterface $em,
48     CsrfTokenManagerInterface $csrf
49 ): JsonResponse {
50     // 1) On récupère le JSON envoyé par le front
51     $data = json_decode($req->getContent(), true) ?? [];
52     $token = (string)($data['_token'] ?? '');
53
54     // 2) Sécurité : vérification du token CSRF
55     if (!$csrf->isTokenValid(new CsrfToken('kanban_order', $token))) {
56         return new JsonResponse(['ok' => false, 'error' => 'bad_csrf'], 400);
57     }
58
59     // 3) Mise à jour des colonnes et des positions
60     $columns = ['todo', 'doing', 'done', 'urgent'];
61     foreach ($columns as $col) {
62         // $data[$col] contient la liste des ID dans cette colonne
63         $ids = array_map('intval', (array)($data[$col] ?? []));
64         foreach ($ids as $i => $id) {
65             // vérification de l'existence de la tâche et de son appartenance à l'utilisateur courant
66             if ($task = $repo->findOneBy(['id' => $id, 'owner' => $this->getUser(),]))
67             {
68                 $task->setStatus($col);
69                 $task->setPosition($i);
70             }
71         }
72     }
73     // 4) On enregistre toutes les modifications en base PostgreSQL
74     $em->flush();
75     // 5) On renvoie une petite réponse JSON
76     return new JsonResponse(['ok' => true]);
```

- #[Route(..., methods:['POST'])]

Définit l'URL /kanban/save-order. Méthode POST uniquement (on modifie des données).

- Paramètres de la méthode :

Request \$req — lire le corps du POST (JSON).

TaskRepository \$repo — accéder aux tâches (Doctrine).

EntityManagerInterface \$em — enregistrer les changements (flush).

CsrfTokenManagerInterface \$csrf — vérifier le token CSRF.

- `$data = json_decode($req->getContent(), true) ?? [];`

Convertit le JSON reçu en tableau PHP. S'il n'y a rien, on a un tableau vide.

- `$token = (string)($data['_token'] ?? "");`

Récupère le token CSRF envoyé par le front.

- `if (!$csrf->isTokenValid(new CsrfToken('kanban_order', $token))) { ... }`

Sécurité : si le token est invalide → on refuse (HTTP 400).

Objectif : empêcher qu'un autre site pousse des actions à l'insu de l'utilisateur (CSRF).

- `$columns = ['todo', 'doing', 'done', 'urgent'];`

Liste des statuts/colonnes gérées par mon Kanban.

- `foreach ($columns as $col) { ... }`

On parcourt chaque colonne envoyée par le front.

- `$ids = array_map('intval', (array)($data[$col] ?? []));`

Pour chaque colonne, on récupère la liste des identifiants envoyés par le front, on les convertit en entiers, puis on met à jour uniquement les tâches existantes appartenant à l'utilisateur courant.

- `foreach ($ids as $i => $id) { if ($task = $repo->findOneBy(['id' => $id, 'owner' => $this->getUser(),])) { ... } }`

Pour chaque ID : Doctrine retrouve l'entité Task en base.

- `$task->setStatus($col);`

Règle métier : la carte prend le statut de la colonne.

- `$task->setPosition($i);`

Règle métier : la position correspond à l'index dans la colonne (0,1,2...).

- `$em->flush();`

Doctrine envoie en base toutes les modifications accumulées (UPDATE), via DBAL/PDO, de façon groupée et sûre.

- `return new JsonResponse(['ok' => true]);`

Réponse JSON claire pour le front : tout s'est bien passé.

Ce que montre ce composant métier

- Entrée : JSON du navigateur (ordre des cartes, token CSRF).
- Contrôle : vérification CSRF.
- Logique métier : statut = colonne, position = ordre visuel.
- Accès données : `TaskRepository->findOneBy()` et `flush()` (Doctrine ORM).
- Sortie : JSON de confirmation, sans rechargement.

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

J'ai utilisé le framework Symfony avec PHP , le moteur Doctrine ORM pour manipuler les entités Task, et le composant CSRF de Symfony pour sécuriser la requête.

Le traitement s'effectue côté serveur via un contrôleur et la réponse est renvoyée en JSON sans rechargement de page.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre de formation DAWAN*

Chantier, atelier, service ► Projet personnel réalisé en cours de formation

Période d'exercice ► Du 01/09/2025 au 02/10/2025

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°4 ► CP 8 Documenter le déploiement d'une application Web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

1) Prérequis

- PHP ≥ 8.2
- Composer
- Symfony CLI
- PostgreSQL 16 (local ou via Docker Compose)
- Git (pour cloner le projet)

2) Récupérer le projet

```
git clone <mon-repo-git>
cd <mon-repo>
composer install
```

3) Variables d'environnement

Créer un fichier .env.local à la racine :

```
APP_ENV=dev
# Secret d'app (généré automatiquement si absent)
APP_SECRET=changeme
# Connexion PostgreSQL
DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:5432/symfony_todo?charset=utf8"
# Messenger (par défaut sur Doctrine, pas nécessaire pour lancer l'app)
MESSENGER_TRANSPORT_DSN=doctrine://default?auto_setup=0
# Mailer en local (facultatif)
MAILER_DSN=smtp://127.0.0.1:1025
```

Remarque : les vraies infos sensibles ne doivent jamais être committées. .env.local reste en local.

4) Démarrer la base de données

PostgreSQL via Docker Compose

```
docker compose up -d
```

Attendre que PostgreSQL écoute sur 127.0.0.1:5432

5) Créer la base et le schéma

```
php bin/console doctrine:database:create
```

```
php bin/console doctrine:migrations:migrate -n
```

6) Lancer le serveur Symfony

```
symfony server:start -d
```

L'application est accessible sur l'URL fournie par Symfony (ex. <http://127.0.0.1:8000>).

7) Logs & arrêt

- Logs applicatifs : `var/log/`
- Arrêter le serveur Symfony :

```
symfony server:stop
```

- Arrêter la base (Docker) :

```
docker compose down
```

2. Précisez les moyens utilisés :

- Symfony CLI pour lancer le serveur local (`symfony server:start` et `symfony server:stop`).
- Docker Compose pour exécuter la base de données PostgreSQL (`docker compose up -d` et `docker compose down`).
- Fichier `.env.local` pour configurer les variables d'environnement (connexion à la base de données, `APP_SECRET`, etc.).
- Terminal (WSL/Debian) pour exécuter les commandes `php bin/console doctrine:database:create` et `doctrine:migrations:migrate`.
- Navigateur web (<http://127.0.0.1:8000>) pour vérifier le bon fonctionnement de l'application.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre de formation DAWAN*

Chantier, atelier, service ► Projet personnel réalisé en cours de formation

Période d'exercice ► Du 01/09/2025 au 02/10/2025

5. Informations complémentaires (facultatif)

Le dépôt GitHub de ce projet est accessible à l'adresse suivante :

<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

<https://symfonytodolist.alwaysdata.net>

DOSSIER PROFESSIONNEL (DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

DOSSIER PROFESSIONNEL (DP)

Déclaration sur l'honneur

Je soussigné(e) Nadezhda Thouvenin Malygina ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à *Roques* le *14/11/2025*

pour faire valoir ce que de droit.

Signature :

DOSSIER PROFESSIONNEL (DP)

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL (DP)

ANNEXES

(Si le RC le prévoit)

